

TOKYO, JAPAN / DECEMBER 11-13, 2025

Debugging RISC-V Kernel Crashes: KDUMP, vmcore, and Crash Utility Enhancements

Austin Kim <austindh.kim@gmail.com>



東京 2025
LINUX PLUMBERS CONFERENCE
TOKYO, JAPAN / DEC. 11-13, 2025

Who I am

- RISC-V enthusiast at LG Electronics
- Author of Reverse Engineering on Armv8-A Systems
- Chipset bring-up for smartphones and automotive systems
- Main Interests: RISC-V, Linux kernel, crash debugging, and Arm architecture

Conference Goal

- Show how to enable KDUMP on RISC-V hardware (VisionFive2 example).
- Explain the patches needed to extract vmcore.
- Demonstrate vmcore analysis using the crash utility.
- Share problems we found and future work.

Background

- Why the debugging environment is important?
  - During RISC-V board bring-up, we go through exception or crashes
  - On real RISC-V boards, kernel logs do not show enough debugging details.
- A vmcore is needed to understand what happens inside the kernel during a crash.
  - Linux developers enable KDUMP, extract the vmcore, and analyze it with the crash utility.
  - QEMU is useful, but real hardware such as VisionFive2 needs memory dump for more accurate debugging.

Why Memory dump Matters

- vmcore contains critical signatures, such as:
  - Register values, Page tables, and CSRs
  - Kernel structures and call stacks
  - Memory layout
- Better vmcore support = faster debugging = faster development

Current debugging feature for dump analysis (Chipset vendor)

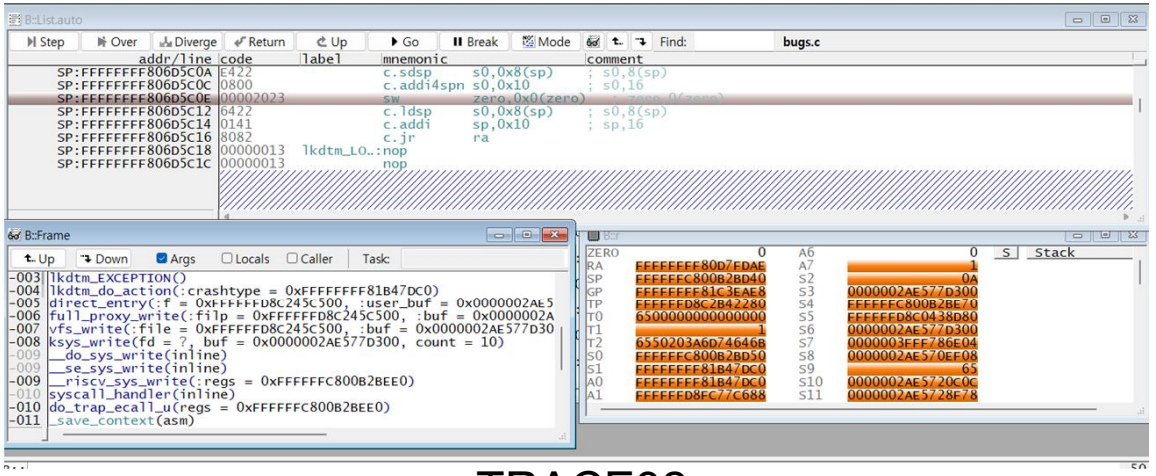
- KDUMP + Ramdump
  - It provides the details on the Linux kernel - call stack and memory contents
  - Chipset vendor (e.g: Qualcomm) supports ramdump
- Crash utility
  - Most used tool to debug kernel structures
  - With the extension of this tool, ftrace log can be extracted from vmcore
- TRACE32
  - Popular tool to inspect the memory contents at architecture level
  - Still used by many BSP engineers in chipset vendor

Case Study: Current debugging feature

- The example of the debug signature (RISC-V)

```
crash> bt
PID: 1490     TASK: ffffffd8c2b42280  CPU: 2    COMMAND: "bash"
 #0 [ffffffc800b2bd40] lkdtm_EXCEPTION at ffffffff806d5c0e
 #1 [ffffffc800b2bd50] lkdtm_do_action at ffffffff80d7fdae
 #2 [ffffffc800b2bd60] direct_entry at ffffffff806d59be
 #3 [ffffffc800b2bd90] full_proxy_write at ffffffff803fc582
 #4 [ffffffc800b2bdd0] vfs_write at ffffffff8020fc70
 #5 [ffffffc800b2be70] ksys_write at ffffffff80210042
 #6 [ffffffc800b2beb0] __riscv_sys_write at ffffffff802100e6
 #7 [ffffffc800b2bec0] $xrv64i2p1_m2p0_a2p1_c2p0_zicsr2p0_zifencei2p0_zihintpause2
crash> log
[    0.000000] Linux version 6.6.20+ (mikeseo@mikeseo-0-1) (riscv64-linux-gnu-gcc
42) #13 SMP Mon Aug 19 12:58:52 KST 2024
[    0.000000] Machine model: StarFive VisionFive 2 v1.3B
[    0.000000] SBI specification v1.0 detected
[    0.000000] SBI implementation ID=0x1 Version=0x10002
[    0.000000] SBI TIME extension detected
```

Crash utility



TRACE32

ftrace

```
logd.reader.per-761  [000]  1167.307538: irq_handler_entry:  irq=37 name=1000000.pinctrl handler=key_down_handle_irq
logd.reader.per-761  [000]  1167.307547: irq_handler_entry:  irq=121 name=volume_up handler=gpio_keys_gpio_isr
logd.reader.per-761  [000]  1167.307557: irq_handler_entry:  irq=37 name=1000000.pinctrl handler=key_down_handle_irq
logd.reader.per-761  [000]  1167.307572: irq_handler_entry:  irq=121 name=volume_up handler=gpio_keys_gpio_isr
logd.reader.per-761  [000]  1167.307582: irq_handler_entry:  irq=37 name=1000000.pinctrl handler=key_down_handle_irq
logd.reader.per-761  [000]  1167.307589: irq_handler_entry:  irq=121 name=volume_up handler=gpio_keys_gpio_isr
logd.reader.per-761  [000]  1167.307603: irq_handler_entry:  irq=37 name=1000000.pinctrl handler=key_down_handle_irq
logd.reader.per-761  [000]  1167.307612: irq_handler_entry:  irq=121 name=volume_up handler=gpio_keys_gpio_isr
logd.reader.per-761  [000]  1167.307622: irq_handler_entry:  irq=37 name=1000000.pinctrl handler=key_down_handle_irq
logd.reader.per-761  [000]  1167.307629: irq_handler_entry:  irq=121 name=volume_up handler=gpio_keys_gpio_isr
```

Case Study: IRQ Storm in RISC-V board

- The ftrace message (from vmcore)

```
logd.reader.per-761  [000]  1167.307538: irq_handler_entry:   irq=37 name=1000000.pinctrl handler=key_down_handle_irq
logd.reader.per-761  [000]  1167.307547: irq_handler_entry:   irq=121 name=volume_up handler=gpio_keys_gpio_isr
logd.reader.per-761  [000]  1167.307557: irq_handler_entry:   irq=37 name=1000000.pinctrl handler=key_down_handle_irq
logd.reader.per-761  [000]  1167.307572: irq_handler_entry:   irq=121 name=volume_up handler=gpio_keys_gpio_isr
logd.reader.per-761  [000]  1167.307582: irq_handler_entry:   irq=37 name=1000000.pinctrl handler=key_down_handle_irq
logd.reader.per-761  [000]  1167.307589: irq_handler_entry:   irq=121 name=volume_up handler=gpio_keys_gpio_isr
logd.reader.per-761  [000]  1167.307603: irq_handler_entry:   irq=37 name=1000000.pinctrl handler=key_down_handle_irq
logd.reader.per-761  [000]  1167.307612: irq_handler_entry:   irq=121 name=volume_up handler=gpio_keys_gpio_isr
logd.reader.per-761  [000]  1167.307622: irq_handler_entry:   irq=37 name=1000000.pinctrl handler=key_down_handle_irq
logd.reader.per-761  [000]  1167.307629: irq_handler_entry:   irq=121 name=volume_up handler=gpio_keys_gpio_isr
logd.reader.per-761  [000]  1167.307639: irq_handler_entry:   irq=37 name=1000000.pinctrl handler=key_down_handle_irq
logd.reader.per-761  [000]  1167.307646: irq_handler_entry:   irq=121 name=volume_up handler=gpio_keys_gpio_isr
logd.reader.per-761  [000]  1167.307655: irq_handler_entry:   irq=37 name=1000000.pinctrl handler=key_down_handle_irq
logd.reader.per-761  [000]  1167.307663: irq_handler_entry:   irq=121 name=volume_up handler=gpio_keys_gpio_isr
...
logd.reader.per-761  [000]  1167.550325: irq_handler_entry:   irq=121 name=volume_up handler=gpio_keys_gpio_isr
logd.reader.per-761  [000]  1167.550334: irq_handler_entry:   irq=37 name=1000000.pinctrl handler=key_down_handle_irq
logd.reader.per-761  [000]  1167.550342: irq_handler_entry:   irq=121 name=volume_up handler=gpio_keys_gpio_isr
logd.reader.per-761  [000]  1167.550353: irq_handler_entry:   irq=35 name=wdog_reset handler=wdog_reset_handler
```

Main Challenge

- KDUMP on RISC-V is not fully supported
- Two major problems appeared:
  - Cannot extract vmcore without specific patches
  - crash utility does not show correct debugging information for RISC-V

Problem 1: vmcore Not Generated

- On VisionFive2, KDUMP did not generate a vmcore.
- vmcore was generated only after applying a specific patch set.
- This shows that default environments may not support crash dump on some RISC-V boards.

Problem 1: vmcore Not Generated

- 1st patch

```
diff --git a/arch/riscv/kernel/setup.c b/arch/riscv/kernel/setup.c
index aac853ae4..f1cffdc7d 100644
--- a/arch/riscv/kernel/setup.c
+++ b/arch/riscv/kernel/setup.c
@@ -173,6 +173,13 @@ static void __init init_resources(void)
        if (ret < 0)
                goto error;

+#ifdef CONFIG_KEXEC_CORE
+       if (crashk_res.start != crashk_res.end) {
+               ret = add_resource(&iomem_resource, &crashk_res);
+               if (ret < 0)
+                       goto error;
+       }
+#endif
 #ifdef CONFIG_CRASH_DUMP
        if (elfcorehdr_size > 0) {
                elfcorehdr_res.start = elfcorehdr_addr;
```

```
crash> p crashk_res
crashk_res = $2 = {
  start = 0xf0000000,
  end = 0xffffffff,
  name = 0xffffffff8152bad8 "Crash kernel",
  flags = 0x81000200,
  desc = 0x1,
  parent = 0xffffffd8fc7b5240,
  sibling = 0x0,
  child = 0x0
}
```

Problem 1: vmcore Not Generated

- 2nd patch

```
diff --git a/arch/riscv/kernel/machine_kexec.c b/arch/riscv/kernel/machine_kexec.c
index f6c7135b00d7..d7ddf4d2b243 100644
--- a/arch/riscv/kernel/machine_kexec.c
+++ b/arch/riscv/kernel/machine_kexec.c
@@ -149,20 +149,12 @@ static void machine_kexec_mask_interrupts(void)

        for_each_irq_desc(i, desc) {
                struct irq_chip *chip;
-               int ret;

                chip = irq_desc_get_chip(desc);
                if (!chip)
                        continue;

-               /*
-                * First try to remove the active state. If this
-                * fails, try to EOI the interrupt.
-                */
-               ret = irq_set_irqchip_state(i, IRQCHIP_STATE_ACTIVE, false);
-
-               if (ret && irqd_irq_inprogress(&desc->irq_data) &&
-                   chip->irq_eoi)
+               if (chip->irq_eoi && irqd_irq_inprogress(&desc->irq_data))
                        chip->irq_eoi(&desc->irq_data);

                if (chip->irq_mask)
```

Problem 2: Wrong crash Utility Output

- Even with vmcore extracted, crash utility did not print structures specific to RISC-V.
- RISC-V support in crash utility is incomplete.
  - Wrong output for: pgd, pmd, pte
  - This makes memory debugging difficult

Why This Happens

- crash utility needs more RISC-V-specific support.
  - Page table formats and flag bits are different.
  - Some CSR values are missing in vmcore header.
- Debugging becomes incomplete or misleading.

Current status of crash utility to support RISC-V

- The number of commit specific to RISC-V

```
crash$ git log --oneline arm64.c | wc -l
130
crash$ git log --oneline riscv64.c | wc -l
19
crash$ git log --oneline x86_64.c | wc -l
118

crash$ git log --oneline riscv64.c
536a0d7 RISCV64: fix wrong information of PMD and PTE - SA39(4K page) support
b3c8d98 RISCV64: Add 'PAGE DIRECTORY' property to the 'vtop' command
974868c RISCV64: Add more system properties to the 'mach' command
a694962 RISCV64: Add per-cpu overflow stacks support
12fbed3 RISCV64: Add per-cpu IRQ stacks support
[...]
67216c7 RISCV64: Add irq command support
b410e14 RISCV64: Add 'dis' command support
e7119d2 RISCV64: Make crash tool enter command line and support some commands
```

Current status of crash utility to support RISC-V

- The patch to add more configuration in Linux kernel:

arch/riscv/kernel/vmcore_info.c

```
+void arch_crash_save_vmcoreinfo(void)
+{
+       VMCOREINFO_NUMBER(phys_ram_base);
+
+       vmcoreinfo_append_str("NUMBER(PAGE_OFFSET)=0x%lx\n", PAGE_OFFSET);
+       vmcoreinfo_append_str("NUMBER(VMALLOC_END)=0x%lx\n", VMALLOC_END);
```

```
crash> rd -f 0x2000 0x3000 -a
            2000:  PG_head_mask)=64
            2011:  NUMBER(PAGE_BUDDY_MAPCOUNT_VALUE)=-129
            2038:  NUMBER(PG_hugetlb)=8
            204d:  NUMBER(PAGE_OFFLINE_MAPCOUNT_VALUE)=-257
            2076:  SYMBOL(kallsyms_names)=ffffffff813f0d70
            209e:  SYMBOL(kallsyms_num_syms)=ffffffff813f0d68
            20c9:  SYMBOL(kallsyms_token_table)=ffffffff814b2a68
            20f7:  SYMBOL(kallsyms_token_index)=ffffffff814b2e20
            2125:  SYMBOL(kallsyms_offsets)=ffffffff814b3020
            214f:  SYMBOL(kallsyms_relative_base)=ffffffff814ed610
            217f:  NUMBER(phys_ram_base)=1073741824
            21a0:  NUMBER(PAGE_OFFSET)=0xffffffd800000000
            21c7:  NUMBER(VMALLOC_START)=0xffffffc800000000
            21f0:  NUMBER(VMALLOC_END)=0xffffffd800000000
            2217:  NUMBER(VA_BITS)=39
            222a:  NUMBER(VMEMMAP_START)=0xffffffc700000000
            2253:  NUMBER(VMEMMAP_END)=0xffffffc800000000
            227a:  NUMBER(MODULES_VADDR)=0xffffffff01d08000
            22a3:  NUMBER(MODULES_END)=0xffffffff80000000
            22ca:  NUMBER(KERNEL_LINK_ADDR)=0xffffffff80000000
            22f6:  NUMBER(va_kernel_pa_offset)=0xffffffff3fe00000
            2325:  CRASHTIME=1724310788
```

Case: fix wrong information of PMD and PTE

https://github.com/crash-utility/crash/commit/536a0d78f1fa0838007806e222bf0eda08b83ecf
RISCV64: fix wrong information of PMD and PTE - SA39(4K page) support

When running 'vtop address' command in RISC-V based vmcore, the base address
without applying offset for PMD and PTE is displayed. This is not correct
address for PMD and PTE.

With this patch, the actual address for PMD and PTE with the valid offset
is printed:

<Before>
```
crash> vtop 0xffffffff81c4f000
VIRTUAL           PHYSICAL
ffffffff81c4f000  41e4f000

PAGE DIRECTORY: ffffffff81c4f000
  PGD: ffffffff81c4fff0 => 4fffe801
  PMD: 000000013fffa000 => 00000000107800e7
  PTE: 41e00000 => ffffffff8166fea0

    PTE        PAGE: 0000000000000000 not present
```

<After>
```
crash> vtop 0xffffffff81c4f000
VIRTUAL           PHYSICAL
ffffffff81c4f000  41e4f000

PAGE DIRECTORY: ffffffff81c4f000
  PGD: ffffffff81c4fff0 => 4fffe801
  PMD: 000000013fffa070 => 00000000107800e7
  PTE: 41e00278 => ffffffff8166fea0

    PTE        PAGE: 0000000000000000 not present
```

Future Work

- We plan to:
  - Enable KDUMP on other RISC-V boards (e.g.: Andes, T-head and Beagle Board-V).
  - Test and fix new issues.
  - Improve crash utility for RISC-V vmcore support.
  - Add more fields (e.g., satp, sstatus) into vmcore header.

Community Feedback Needed

- We want help from the community:
  - Add more RISC-V register data into vmcore header.
  - Enable KDUMP on more popular RISC-V boards.
  - Improve crash utility to display correct page table and low-level data.



東京 2025
LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025

